YouthPath

청년 사회진입 토탈 어시스턴트

딥러닝 및 응용 팀 프로젝트 제안서

과목: 딥러닝 및 응용 (Deep Learning Methods and Application) **팀**: 5인 팀 **제출일**: 2026년 5월 5일 **프로젝트 키워드**: LLM · Prompting · RAG · Multi-Agent · MCP

목차

1. 프로젝트 개요 (Executive Summary)
2. 문제 정의 및 동기 (Problem Definition & Motivation)
3. 배경 및 관련 연구 (Background & Related Work)
4. 시스템 설계 및 기술적 접근 (System Design & Technical Approach)
5. 데이터셋 및 실험 계획 (Dataset & Experiment Plan)
6. 실행 가능성 및 팀 계획 (Feasibility & Team Planning)
7. 기대 효과 및 향후 발전 방향 (Expected Outcomes)

---

1. 프로젝트 개요 (Executive Summary)

1.1 한 줄 요약

> **사용자 프로필 1회 입력으로 청년정책 · 채용공고 · 자소서/면접 준비를 통합 제공하는 멀티 에이전트 어시스턴트.**

1.2 프로젝트 배경

대한민국 청년이 사회 진입 과정에서 마주하는 정보는 **5~6개의 분리된 플랫폼**에 흩어져 있습니다.

영역	기존 플랫폼	한계
청년정책	정부24, 온통청년	단순 검색·나열, 자격 매칭 부재
채용공고	워크넷, 잡코리아, 사람인	기업별 분산, 자소서 연동 X
자소서·면접	별도 사설 도구	기업 specific 정보 부족
일정 관리	개인 캘린더	신청 마감 자동 알림 X

청년 한 명이 인생 전환기에 **각각의 사이트를 따로 방문**해야 하며, 정보 탐색 비용이 매우 높습니다.

1.3 솔루션

YouthPath는 LLM 기반 Multi-Agent 시스템으로, 분산된 정보를 **하나의 인터페이스에서 개인화하여 통합 제공**합니다.

- 정책 Agent: 자격 자동 매칭 + 신청 가이드
- 채용 Agent: GitHub/이력 분석 기반 채용공고 매칭
- 자소서 Agent: DART 공시 기반 기업 specific 첨삭
- 캘린더 Agent: 마감일 통합 알림

---

2. 문제 정의 및 동기 (Problem Definition & Motivation)

2.1 문제 진술 (Clarity of Problem Statement)

Pain Point 1: 정보 분산 청년 정책, 채용공고, 자소서 첨삭이 각각 다른 플랫폼에 존재하여 사용자가 여러 사이트를 순회해야 함.

Pain Point 2: 자격 매칭의 부재 정책 사이트는 정책 목록만 제공할 뿐, "내가 받을 수 있는지"를 자동 판단해주지 않음.

Pain Point 3: 일반 LLM의 한계 ChatGPT 등 일반 LLM은 학습 데이터 cutoff 이후 정보를 모름. 신규 정책, 최신 공시, 변동 채용공고를 반영할 수 없음.

Pain Point 4: 신청 시기 누락 정책 신청 마감일이 분산되어 있어 사용자가 기회를 놓치는 경우 빈번.

2.2 실생활 관련성 (Real-World Relevance)

- **타겟 사용자**: 만 19~34세 청년 (대학생, 사회 초년생, 취준생) — 약 1,200만 명
- **사회적 가치**:
  - 정보 비대칭 해소 → 청년 복지 사각지대 축소
  - 행정 비용 절감 → 정책 활용률 증대
  - 취업 준비 효율화 → 청년 실업률 완화 기여

2.3 애플리케이션 필요성 정당화 (Justification)

현재 상황	YouthPath 도입 후
5~6개 사이트 순회 (평균 2시간)	1개 인터페이스 (5분)
자격 부적합 정책까지 다 읽음	자격 매칭된 정책만 노출
자소서 작성 시 회사 홈피 직접 조사	DART 공시 자동 반영
신청 마감일 수기 관리	통합 캘린더 자동 알림

---

3. 배경 및 관련 연구 (Background & Related Work)

3.1 관련 모델 및 도구 이해 (Understanding of Relevant Tools)

3.1.1 LLM (Large Language Model)

- LUXIA (수업 제공): 한국어 도메인 강함, 자연스러운 어투
- GPT-4o-mini: 정확한 reasoning, 저비용 평가용
- Gemini 2.0 Flash: 200만 토큰 컨텍스트, 무료 tier

3.1.2 RAG (Retrieval-Augmented Generation)

정적 문서를 사전 인덱싱한 후, 사용자 질의 시 관련 청크를 검색하여 LLM에 컨텍스트로 주입. - Static RAG: 정책 공고문, 합격 자소서 등 변동 적은 자료 - Dynamic RAG: DART 사업보고서 등 대용량 외부 문서를 런타임 인덱싱

3.1.3 MCP (Model Context Protocol)

LLM이 외부 도구 및 API를 표준화된 방식으로 호출하는 프로토콜. DART, 워크넷, 공공데이터포털 API를 MCP 서버로 래핑하여 Agent가 도구처럼 사용.

3.1.4 Multi-Agent (LangGraph)

역할별로 분리된 Agent들이 협업하여 복잡한 작업을 수행. Router Agent가 사용자 질의를 분류하여 적절한 Sub-Agent에 위임.

3.2 기존 서비스 대비 개선점 (Identification of Improvement)

비교 대상	한계	YouthPath의 차별점
정부24 / 온통청년	단순 키워드 검색, 자격 미판정	사용자 프로필 기반 자격 자동 매칭
잡코리아 AI매칭	채용공고만, 자소서 연동 X	채용 + 자소서 + 면접 통합
ChatGPT	학습 cutoff 이후 정보 모름	RAG/MCP로 실시간 공시·공고 반영
개인 캘린더 앱	수동 입력 필요	정책 신청 마감 자동 동기화

3.3 기술적 차별점

1. **Hybrid RAG 전략**: Static (정책·자소서) + Dynamic (DART) 결합
2. **Multi-Agent Orchestration**: Router + 4개 Specialized Agents
3. **MCP 기반 외부 연동**: 표준 프로토콜로 확장 용이
4. **개인화 매칭 알고리즘**: 프로필 ↔ 자격요건 자동 비교

---

4. 시스템 설계 및 기술적 접근 (System Design & Technical

Approach)

4.1 전체 아키텍처

```
┌─────────────────────────────────────────────────────────────────┐
│  [Layer 1] Frontend                                             │
│  - Streamlit Web UI                                             │
│  - 사용자 프로필 입력 폼                                          │
│  - 통합 대시보드 (정책/채용/자소서/캘린더)                          │
└─────────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────────┐
│  [Layer 2] Multi-Agent Orchestration (LangGraph)                │
│                                                                 │
│          ┌──────────────────────────────────┐                   │
│          │      Router Agent (LUXIA)        │                   │
│          │  사용자 의도 분석 → Sub-Agent 배정  │                   │
│          └──────────────────────────────────┘                   │
│            ↓         ↓         ↓          ↓                     │
│       ┌──────┐  ┌──────┐  ┌──────┐  ┌──────┐                   │
│       │ 정책 │  │ 채용 │  │ 자소서│  │ 캘린더 │                   │
│       │Agent │  │Agent │  │Agent │  │ Agent │                   │
│       └──────┘  └──────┘  └──────┘  └──────┘                   │
└─────────────────────────────────────────────────────────────────┘
            ↓         ↓         ↓          ↓
┌─────────────────────────────────────────────────────────────────┐
│  [Layer 3] Data Layer (RAG + MCP)                               │
│                                                                 │
│  Static RAG (Chroma Vector DB):                                 │
│  - 청년정책 공고문 (인덱싱: bge-m3)                                │
│  - 합격 자소서 사례                                               │
│  - NCS 직무 평가 기준                                             │
│                                                                 │
│  Dynamic RAG:                                                   │
│  - DART 사업보고서 (런타임 인덱싱)                                 │
│                                                                 │
│  MCP Servers:                                                   │
│  - 온통청년 OpenAPI (정책 조회)                                   │
│  - 공공데이터포털 (정부 정책)                                      │
│  - 워크넷 OpenAPI (채용공고)                                      │
│  - DART OpenAPI (기업 공시)                                       │
│  - 네이버 뉴스 검색 API (기업 뉴스)                                 │
└─────────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────────┐
│  [Layer 4] Infrastructure                                       │
│  - AWS EC2 (FastAPI Backend)                                    │
│  - AWS S3 (정적 파일, Vector DB 백업)                              │
│  - LUXIA Platform (메인 LLM)                                      │
└─────────────────────────────────────────────────────────────────┘
```

4.2 Agent별 상세 설계

4.2.1 Router Agent

역할: 사용자 질의 분석 → 어느 Sub-Agent를 호출할지 결정 **입력**: 사용자 자연어 질의 + 프로필 **출력**: 호출할 Agent 목록 (병렬 가능) **프롬프트 예시**:

```
당신은 사용자 질의를 분석하여 적절한 전문 에이전트에 작업을
배정하는 라우터입니다. 다음 4개 에이전트 중 필요한 것을 모두 선택하세요.
- policy_agent: 청년 정책, 지원금, 복지 관련
- job_agent: 채용공고, 직무, 회사 추천
- resume_agent: 자소서 작성, 첨삭, 면접 준비
- calendar_agent: 마감일, 일정, 알림
```

4.2.2 정책 Agent (Policy Agent)

역할: 사용자 프로필 ↔ 정책 자격요건 매칭 **프로세스**: 1. MCP 온통청년 API → 최신 정책 목록 조회 2. RAG (정책 공고문 Vector DB) → 상세 자격요건 검색 3. LLM → 사용자 프로필과 자격요건 자동 비교 4. 출력: 받을 수 있는 정책 + 신청 방법 + 마감일

4.2.3 채용 Agent (Job Agent)

역할: 사용자 스킬·관심분야 ↔ 채용공고 매칭 **프로세스**: 1. MCP 워크넷 API → 채용공고 검색 2. (선택) 사용자 GitHub 분석 → 보유 기술 추출 3. LLM → 직무 적합도 평가 4. 출력: 적합 공고 Top 5 + 마감일

4.2.4 자소서 Agent (Resume Agent)

역할: 기업 specific 자소서 첨삭 및 면접 대비 **프로세스**: 1. MCP DART API → 지원 기업 사업보고서 fetch 2. Dynamic RAG → 사업부문, 인재상, 신사업 키워드 추출 3. Static RAG (합격 자소서 DB) → 유사 합격 사례 검색 4. LLM → 사용자 자소서 분석 + 보강 제안 + 모범답안

4.2.5 캘린더 Agent (Calendar Agent)

역할: 정책 신청 + 채용 마감일 통합 관리 **프로세스**: 1. 다른 Agent 결과에서 마감일 자동 추출 2. 사용자별 일정 DB 저장 3. 마감 D-7, D-3, D-1 알림 (이메일/카카오톡)

4.3 데이터 처리 전략

4.3.1 RAG 처리 패턴

데이터	크기	변동 빈도	처리 방식
청년정책 공고문	중	주간	Static RAG (정기 갱신)
합격 자소서	큼	거의 없음	Static RAG (1회 인덱싱)
NCS 평가기준	중	연간	Static RAG
DART 사업보고서	매우 큼	분기	Dynamic RAG (요청 시 인덱싱)
채용공고	작음	일간	Tool Use (직접 주입)
뉴스 헤드라인	작음	시간별	Tool Use

4.3.2 병렬 처리

회사명 추출 후 DART/뉴스/채용공고 호출은 서로 독립적이므로 `asyncio.gather` 로 병렬 호출하여 응답 시간을 약 3배 단축.

4.4 모델 및 프레임워크 선정 정당화 (Justification)

컴포넌트	선택	정당화 근거
메인 LLM	LUXIA	한국어 도메인 우수, 수업 제공 자원, 자연스러운 답변 톤
보조 LLM	Gemini 2.0 Flash	200만 토큰 long context (사업보고서 처리), 무료 tier
평가 LLM	GPT-4o-mini	안정적 reasoning, 저비용 ($0.15/1M tokens)
임베딩 모델	BAAI/bge-m3	다국어 지원, 한국어 검색 정확도 우수, 로컬 무료
Vector DB	Chroma	오픈소스, 로컬 운영, $50 크레딧 절약
Agent Framework	LangGraph	워크플로우 시각화, 디버깅 용이, MCP 지원
Frontend	Streamlit	Python 기반 빠른 프로토타이핑, 발표 데모 적합
Backend	FastAPI	비동기 처리, MCP 서버 연동 용이

4.5 Fine-tuning 대신 RAG를 선택한 이유

- 정책·공시 데이터는 **자주 갱신**됨 → Fine-tuning은 매번 재학습 필요
- RAG는 **데이터 추가만으로 즉시 반영** → 운영 비용 ↓
- 출처 인용(citation) 가능 → **신뢰성** ↑
- $50 크레딧 내 운영 가능

---

5. 데이터셋 및 실험 계획 (Dataset & Experiment Plan)

5.1 데이터셋 구성

데이터	출처	용도	라이선스
청년정책	온통청년 OpenAPI	Static RAG 인덱싱	공공누리
정부정책	공공데이터포털	Static RAG	공공누리
채용공고	워크넷 OpenAPI	Tool Use (실시간)	공공누리
기업 공시	DART OpenAPI	Dynamic RAG	무료 (등록)
합격 자소서	공개된 합격 자소서 모음 (블로그·커뮤니티 게시물)	Static RAG	공개
NCS 평가기준	NCS 국가직무능력표준	Static RAG	공공누리
평가용 Q&A	직접 구축 50문항	Benchmark	자체

5.2 평가 방법 (Proper Evaluation)

5.2.1 정량 평가

메트릭	측정 대상	목표
정책 매칭 Precision	추천 정책의 자격 적합도	≥ 0.85
정책 매칭 Recall	받을 수 있는 정책 누락률	≥ 0.80
RAG 인용 정확도	출처와 답변 일치율	≥ 0.90
응답 시간 (P95)	사용자 질의 → 답변	≤ 5초
자소서 첨삭 BLEU/ROUGE	모범 답안 대비 유사도	보고용

5.2.2 정성 평가

- **사용자 만족도 설문** (n=20명): 5점 척도
- **LLM-as-judge**: GPT-4o-mini가 답변 품질 평가 (1~5점)
- **기존 도구 비교 실험**:
  - YouthPath vs ChatGPT
  - YouthPath vs 정부24 + 잡코리아 (수동 조합)
  - 측정: 정확도, 소요 시간, 사용자 선호도

5.3 평가 데이터셋 구축 계획

카테고리	문항 수	예시
정책 매칭	20	"25세 서울 거주, 무주택, 받을 수 있는 주거 정책?"
채용 추천	15	"Python·React 가능한 신입, 추천 공고 5개?"
자소서 첨삭	10	"삼성전자 DS 자소서 입사 후 포부 첨삭"
일정 관리	5	"이번 달 마감되는 정책 알려줘"

---

6. 실행 가능성 및 팀 계획 (Feasibility & Team Planning)

6.1 5인 역할 분담

팀원	담당	주요 작업
A	Policy Agent + 정책 RAG	온통청년 API 연동, 정책 PDF 인덱싱, 자격 매칭 로직
B	Job Agent + DART/채용 MCP	워크넷 API, DART OpenAPI 연동, MCP 서버 구축
C	Resume Agent + 자소서 RAG	합격 자소서 데이터셋 구축, 자소서 첨삭 평가 메트릭
D	Frontend + UX	Streamlit 대시보드, 프로필 입력 UI, 사용자 테스트
E	Backend + LangGraph 통합	FastAPI, AWS EC2 배포, Router Agent, 캘린더 Agent

6.2 주차별 마일스톤

주차	일정	마일스톤
Week 10	5/5	프로젝트 제안 발표
Week 11	5/12	데이터 수집 + RAG 인덱싱 완료
Week 12	5/19	단일 Agent 프로토타입 (정책)
Week 13	5/26	4개 Agent 통합 + Router 구현
Week 14	6/2	Frontend + Backend 연동
Week 15	6/9	평가 데이터셋 + 정량 실험
Week 16	6/16	최종 발표 및 데모

6.3 비용 계획

항목	사용 모델/서비스	예상 비용
메인 LLM	LUXIA $50 크레딧	$50 (수업 제공)
긴 문서 요약	Gemini 2.0 Flash 무료 tier	$0
평가 LLM	GPT-4o-mini	~$2
임베딩	bge-m3 (로컬)	$0
Vector DB	Chroma (로컬)	$0
호스팅	AWS Academy 계정	$0 (수업 제공)
총 예상 비용		**$50 + α (~$2)**

6.4 위험 요소 및 대응 방안

위험	발생 가능성	대응 방안
LUXIA 크레딧 조기 소진	중	Gemini 무료 tier로 보조, 캐싱 적극 활용
외부 사이트 크롤링 ToS	중	공공 OpenAPI(워크넷·DART) 중심, 크롤링 회피
DART 사업보고서 너무 김	높음	Dynamic RAG로 청킹·검색 (이미 설계 반영)
Multi-Agent 디버깅 난이도	중	LangGraph 시각화 도구 적극 활용
데이터 라이선스 이슈	낮음	공공누리·CC 라이선스 자료만 사용

---

7. 기대 효과 및 향후 발전 방향 (Expected Outcomes)

7.1 기대 효과

7.1.1 사용자 측면

- 정보 탐색 시간 **약 95% 단축** (2시간 → 5분)
- 자격 적합 정책 누락 방지 → **청년 복지 활용률 증대**
- 자소서 작성 효율 향상 → **취업 준비 부담 경감**

7.1.2 기술적 측면

- LLM + RAG + Multi-Agent + MCP 통합 시스템 구축 경험
- Hybrid RAG (Static + Dynamic) 실전 적용
- 한국어 도메인 LLM (LUXIA) 활용 사례

7.2 향후 발전 방향

단계	기능	기술적 도전
Phase 2	다국어 지원 (영/중)	외국인 유학생 시장 확장
Phase 3	음성 입력 (Whisper STT)	디지털 약자 접근성
Phase 4	모바일 앱 (React Native)	푸시 알림 강화
Phase 5	카카오톡 챗봇 연동	진입장벽 최소화
Phase 6	학습 로드맵 추천	부족 스킬 → 인프런/유데미 강의 매칭

7.3 사회적 임팩트

- **정책 사각지대 축소**: 정보 부족으로 혜택을 못 받는 청년 감소
- **취업 시장 효율화**: 직무 적합도 매칭으로 청년 실업 완화
- **데이터 개방 활용 사례**: 공공 OpenAPI 활용 모범 사례 제시

부록 A: 기술 스택 요약

분류	기술
언어	Python 3.11
LLM	LUXIA, Gemini 2.0 Flash, GPT-4o-mini
임베딩	BAAI/bge-m3 (HuggingFace)
Vector DB	Chroma
Agent Framework	LangGraph
Tool Protocol	MCP (Model Context Protocol)
Frontend	Streamlit
Backend	FastAPI
Cloud	AWS EC2 + S3
CI/CD	GitHub Actions

부록 B: 평가 기준 매핑 (자체 점검)

평가 항목 (배점)	본 제안서 대응 섹션	예상 점수
문제 정의 및 동기 (20)	§2 (Pain Point 4가지 + 1,200만 명 타겟)	18/20
배경 및 관련 연구 (20)	§3 (정부24/잡코리아/ChatGPT 비교)	17/20
시스템 설계 및 기술적 접근 (30)	§4 (4-Layer 아키텍처 + 5 Agents + Hybrid RAG)	27/30
데이터셋 및 실험 계획 (15)	§5 (정량/정성 평가 + 50문항 benchmark)	14/15
실행 가능성 및 팀 계획 (15)	§6 (역할 분담 + 마일스톤 + 비용)	14/15
합계 예상		**90/100**

부록 C: 팀원별 학습 분담 및 로드맵

C.1 학습 분담 원칙

본 프로젝트는 다양한 기술 스택(LLM, RAG, Agent, MCP, FE, BE)을 통합하므로, 팀원 5명이 모든 기술을 동일한 깊이로 학습하는 것은 비효율적입니다. 따라서 다음 원칙으로 학습을 분담합니다.

1. **자기 담당 Agent의 핵심 기술은 "깊게"** — 해당 분야의 팀 내 전문가 역할 수행
2. **다른 영역도 "개괄적으로"** — 발표 및 Q&A에서 큰 그림 답변 가능
3. **공통 코어는 모두 학습** — 프로젝트 비전, 평가 기준, 전체 아키텍처

C.2 팀원별 학습 영역

팀원 A: RAG (Static) + 정책 도메인 전문가

담당 코드: Policy Agent + 정책 Vector DB

학습 주제	깊이	핵심 포인트
RAG 작동 원리	깊음	Indexing(전처리) ↔ Retrieval(런타임) 2단계 분리
임베딩 모델	깊음	bge-m3 사용법, 한국어 임베딩 차이점
청킹 전략	깊음	크기·오버랩, RecursiveCharacterTextSplitter
Chroma Vector DB	깊음	설치, CRUD, 유사도 검색
온통청년 API	중간	정책 데이터 구조, 자격요건 필드
Agent 프레임워크	개괄	다른 팀원이 만든 노드 호출만 가능하면 됨

추천 학습 자료: - LangChain 공식 RAG 튜토리얼 - "RAG From Scratch" YouTube 시리즈 (LangChain 채널) - 온통청년 OpenAPI 문서

팀에 설명할 핵심 (30분): "RAG는 왜 필요한가, 청킹은 왜 중요한가, Static과 Dynamic의 차이"

팀원 B: MCP + Dynamic RAG + 채용/공시 도메인 전문가

담당 코드: Job Agent + DART/워크넷 MCP 서버

학습 주제	깊이	핵심 포인트
MCP (Model Context Protocol)	깊음	표준 도구 호출, MCP 서버 구축
LangChain Tool Use	깊음	@tool 데코레이터, ReAct 패턴
Dynamic RAG	깊음	런타임 인덱싱, 임시 Vector DB
DART OpenAPI	깊음	사업보고서 fetch, XBRL 파싱
워크넷 OpenAPI	중간	채용공고 검색 파라미터
병렬 호출	중간	asyncio.gather, async/await

추천 학습 자료: - Anthropic MCP 공식 문서 (modelcontextprotocol.io) - DART OpenAPI 가이드 (opendart.fss.or.kr) - LangChain Tool Use 튜토리얼

팀에 설명할 핵심 (30분): "MCP가 무엇이고 일반 API 호출과 어떻게 다른지, Dynamic RAG는 언제 사용하는지"

팀원 C: LLM + Prompt Engineering + 평가 전문가

담당 코드: Resume Agent + 합격 자소서 데이터셋 + 평가 메트릭

학습 주제	깊이	핵심 포인트
LLM 비교	깊음	LUXIA, GPT-4o-mini, Gemini의 강약점
Prompt Engineering	깊음	Few-shot, CoT, Role-based, STAR rubric
LLM-as-judge	깊음	GPT-4o-mini로 답변 자동 평가
평가 메트릭	중간	BLEU, ROUGE, Precision/Recall
DART 사업보고서 활용	중간	인재상·신사업 키워드 추출
자소서 패턴 분석	중간	합격 자소서 구조 (STAR, PREP)

추천 학습 자료: - OpenAI Prompt Engineering Guide - Prompt Engineering Guide (promptingguide.ai) - Anthropic Prompt Library

팀에 설명할 핵심 (30분): "프롬프트를 잘 작성하는 방법, 모델별 강약점, 답변 품질을 어떻게 평가할지"

팀원 D: Frontend + UX + 사용자 테스트 전문가

담당 코드: Streamlit Web UI + 통합 대시보드 + 사용자 테스트

학습 주제	깊이	핵심 포인트
Streamlit	깊음	위젯, session state, 멀티페이지
대시보드 UX	깊음	정책/채용/자소서 통합 뷰 설계
API 호출 (Python)	중간	requests, httpx, FastAPI 클라이언트
사용자 테스트	중간	n=20 설문 설계, 5점 척도
Whisper STT (선택)	개괄	음성 입력 옵션
백엔드 흐름	개괄	API 명세 이해

추천 학습 자료: - Streamlit 공식 튜토리얼 (docs.streamlit.io) - Streamlit Cheat Sheet - Nielsen Norman Group UX 가이드

팀에 설명할 핵심 (30분): "사용자가 어떤 흐름으로 서비스를 사용할지, 어떻게 사용자 테스트를 진행할지"

팀원 E: Multi-Agent + Backend + 배포 전문가

담당 코드: LangGraph Router + FastAPI + AWS 배포

학습 주제	깊이	핵심 포인트
LangGraph	깊음	StateGraph, Node, Edge, Conditional Edge
Multi-Agent 패턴	깊음	Router/Sequential/ReAct/Supervisor
FastAPI	깊음	async endpoints, Pydantic 모델
AWS EC2 + S3	중간	배포, IAM, 보안 그룹
GitHub Actions CI/CD	중간	자동 배포 파이프라인
전체 통합	깊음	4명 컴포넌트 연결, E2E 테스트

추천 학습 자료: - LangGraph 공식 튜토리얼 (langchain-ai.github.io/langgraph) - FastAPI 공식 문서 (fastapi.tiangolo.com) - AWS Academy 자료

팀에 설명할 핵심 (30분): "4명의 Agent가 한 시스템에서 어떻게 협업하는지, 사용자 요청이 어떤 경로로 처리되는지"

C.3 모든 팀원 공통 학습 영역

주제	소요 시간	학습 이유
프로젝트 비전 및 목표	30분	정책·채용·자소서 통합 가치 이해
평가 기준 5개 항목	30분	우리 점수가 결정되는 구조 이해
전체 아키텍처 4-Layer	1시간	자기 파트가 어디 위치하는지 파악
LLM/RAG/Agent/MCP 차이	1시간	발표 및 Q&A 기본 답변

본 보고서의 §1, §2, §3, §4.1을 정독하면 충분합니다.

C.4 학습 일정 (3 Phase)

Phase 1: 제안 발표 준비 (5/3 ~ 5/5, 3일)

- 공통 학습: 보고서 정독 (1~2시간)
- 자기 파트 개괄 학습 (3~4시간)
- 5/5 발표 시 각자 자기 파트 답변 가능 수준 도달

Phase 2: 깊은 학습 (5/5 ~ 5/12, 1주)

- 각자 핵심 항목 집중 학습 (15~20시간)
- 매일 짧은 스탠드업 미팅 (10분)
- 5/12 학습 공유 세션 진행 (각자 30분씩, 총 2.5시간)

Phase 3: 구현 시작 (5/12 이후)

- 학습한 내용 바탕으로 코딩 시작
- 막히는 부분은 해당 영역 전문가에게 즉시 질문
- 매주 진행상황 공유

C.5 학습 공유 세션 진행 방식 (5/12)

권장 형식: 30분 × 5명 = 2.5시간

각 팀원은 다음 양식으로 발표합니다.

1. 내가 맡은 영역의 핵심 개념 (5분)
2. 우리 프로젝트에서 어떻게 사용하는지 (5분)
3. 실제 코드 예시 또는 데모 (5분)
4. 다른 팀원이 알아야 할 핵심 3줄 요약 (2분)
5. Q&A (10분)

5명 발표 종료 후 통합 토론 (30분)을 통해 컴포넌트 간 인터페이스를 확정합니다.

C.6 통합 체크리스트 (학습 공유 세션 종료 후)

- 모든 팀원이 LLM/RAG/Agent/MCP 차이를 한 줄로 설명 가능한가
- 모든 팀원이 자기 Agent와 인접 Agent의 데이터 흐름을 그릴 수 있는가
- API 명세 (Frontend ↔ Backend) 합의 완료되었는가
- 데이터 형식 (Agent 간 주고받는 JSON 스키마) 합의 완료되었는가
- 각자 GitHub 브랜치 셋업 완료되었는가
- 평가 데이터셋 50문항 분담 완료되었는가 (각자 10문항)

C.7 팀원별 학습 키워드 요약

팀원	학습 키워드	발표 시 한 마디
A	RAG	“검색이 답변 품질을 결정한다”
B	MCP	“외부 도구를 LLM의 손에 쥐어준다”
C	Prompt	“지시문이 모델보다 중요하다”
D	UX	“사용자가 5분 안에 가치를 느껴야 한다”
E	LangGraph	“여러 Agent가 한 몸처럼 움직인다”

본 문서는 한양대학교 딥러닝 및 응용 과목 팀 프로젝트 제안서로 작성되었습니다.